



```
"""
PPO_V3.1: Institutional-Grade Multi-Asset DRL Allocation Pipeline
Enhancements:
- Multi-Stage Tier 2 Circuit Breaker (8% -> 60%, 12% -> 30%, 15% -> 100%
Cash Lockout)
- Trend Regime Gate: 200-day SMA + Negative Momentum Cap (<= 10% per
falling asset)
- Cross-Sectional Momentum Rank (63-day / 3-month Rolling Percentile)
"""

import os
import sys
import random
import warnings
from dataclasses import dataclass, field
from pathlib import Path
from typing import Dict, List, Tuple, Optional

import gymnasium as gym
from gymnasium import spaces
import numpy as np
import pandas as pd
import pandas_datareader.data as web
from scipy import stats
import ta
import torch
import torch.nn as nn
import yfinance as yf
from stable_baselines3 import PPO
from stable_baselines3.common.callbacks import EvalCallback
from stable_baselines3.common.vec_env import DummyVecEnv, SubprocVecEnv,
VecNormalize

warnings.filterwarnings("ignore")

#
=====
====
# SECTION 0: CONFIGURATION & PRE-REGISTRATION PROTOCOL
```

new



```
#
=====

====

@dataclass(frozen=True)
class ProductionConfig:
    core_tickers: Tuple[str, ...] = ("SPY", "QQQ", "DIA", "TLT", "GLD")

    # Financial & Accounting
    initial_amount: float = 1_000_000.0
    transaction_cost_pct: float = 0.0015 # 15 bps
    stress_cost_pct: float = 0.0030 # 30 bps
    min_turnover_threshold: float = 0.01 # 1% No-trade band

    # Tier 2 Risk Overlay (Multi-Stage Circuit Breaker)
    target_annual_vol: float = 0.10 # 10% Target Volatility
    vol_ewma_lambda: float = 0.94 # RiskMetrics standard
    min_exposure: float = 0.20
    max_exposure: float = 1.00
    hysteresis_band: float = 0.05

    # Multi-Stage Thresholds
    dd_stage1: float = 0.08 # DD >= 8% -> Exposure <= 60%
    dd_stage2: float = 0.12 # DD >= 12% -> Exposure <= 30%
    dd_hard_exit: float = 0.15 # DD >= 15% -> Cash 100% Lockout
    trend_cap_weight: float = 0.10 # Cap weight at 10% if asset <
200 SMA and mom < 0

    # Macro Lags
    fed_funds_lag_days: int = 35
    global_fetch_start: str = "1996-01-01"
    global_end: str = "2026-07-29"

    # Hardware & Multiprocessing
    seed: int = 42
    device: str = "cpu"
    max_cpu: int = 8

    # PPO Hyperparameters
    target_rollout: int = 8192
```

[new](#)



```
batch_size: int = 256
total_timesteps: int = 1_000_000
learning_rate: float = 0.0002
n_epochs: int = 10
gamma: float = 0.99
gae_lambda: float = 0.95
clip_range: float = 0.2
ent_coef: float = 0.005
vf_coef: float = 0.5
max_grad_norm: float = 0.5
target_kl: float = 0.02

pre_registered_trials: int = 25

CFG = ProductionConfig()

#
=====
====
# SINGLE (FIXED) CHRONOLOGICAL HOLDOUT SPLIT
# Train [1999-04-01:2019-12-31] -> Val [2020-01-01:2023-12-31] -> Test
[2024-01-01:2026-07-29]
#
=====
====
WALK_FORWARD_FOLDS = [
    ("1999-04-01", "2019-12-31", "2020-01-01", "2023-12-31", "2024-01-01",
"2026-07-29"),
]

TECHNICAL_FEATURES = [
    "RSI_14", "RSI_rel", "MACD_hist_pct", "EMA_12_26_ratio",
    "EMA_50_200_ratio", "price_to_EMA200", "StochRSI_K", "mom_20d",
"mom_63d"
]

CROSS_SECTIONAL_FEATURES = ["ratio_QQQ_SPY", "ratio_SPY_TLT",
"mom_rank_63d"]
MACRO_FEATURES = ["vix_log", "yield_curve_slope", "gold_logret",
"wti_logret", "fed_funds_rate"]
```

[new](#)



```
ALL_FEATURES = TECHNICAL_FEATURES + CROSS_SECTIONAL_FEATURES +
MACRO_FEATURES

#
=====

====
# SECTION 1: DATA ACQUISITION & SYNTHETIC PROXY STITCHING
#
=====

====

def set_global_seed(seed: int):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

def _ensure_naive_datetime(dt_obj):
    dt = pd.to_datetime(dt_obj)
    if isinstance(dt, pd.DatetimeIndex):
        return dt.tz_localize(None) if dt.tz is not None else dt
    elif hasattr(dt, "dt"):
        return dt.dt.tz_localize(None) if dt.dt.tz is not None else dt
    return dt

def _extract_price_df(raw: pd.DataFrame, field: str) -> pd.DataFrame:
    if isinstance(raw.columns, pd.MultiIndex):
        if field in raw.columns.levels[0]:
            df = raw[field].copy()
        elif field in raw.columns.levels[1]:
            df = raw.xs(field, axis=1, level=1).copy()
        else:
            raise KeyError(f"Field '{field}' not found in yfinance
MultiIndex.")
    else:
        df = raw[[field]].copy()
    df.index = _ensure_naive_datetime(df.index)
    return df
```

[new](#)



```
def fetch_and_stitch_universe(tickers: Tuple[str, ...], start_date: str,
end_date: str) -> pd.DataFrame:
    print(f">> Fetching Market Data & Backcasting Inception Gaps:
{start_date} -> {end_date}")
    all_tickers = list(tickers) + ["GC=F", "VUSTX", "^VIX", "^TNX",
"^IRX", "CL=F"]
    raw = yf.download(all_tickers, start=start_date, end=end_date,
progress=False, auto_adjust=False)

    opens = _extract_price_df(raw, "Open")
    closes = _extract_price_df(raw, "Close")
    highs = _extract_price_df(raw, "High")
    lows = _extract_price_df(raw, "Low")
    volumes = _extract_price_df(raw, "Volume")

    opens = opens.fillna(closes).ffill()
    highs = highs.fillna(closes).ffill()
    lows = lows.fillna(closes).ffill()
    closes = closes.ffill()
    volumes = volumes.fillna(1_000_000.0)

    # 1. Backcast GLD with COMEX Gold (GC=F)
    gld_series = closes["GLD"].dropna()
    gld_start = gld_series.index[0]
    gld_anchor = gld_series.loc[gld_start]
    gc_anchor = closes["GC=F"].asof(gld_start)
    if pd.isna(gc_anchor) or gc_anchor <= 0:
        gc_anchor = closes["GC=F"].dropna().iloc[0]
    gld_ratio = gld_anchor / gc_anchor
    pre_gld_mask = closes.index < gld_start

    days_back_gld = (pd.to_datetime(gld_start) -
pd.to_datetime(closes.index[pre_gld_mask])).days
    expense_factors_gld = (1.0 - 0.0040 / 365.25) ** days_back_gld

    closes.loc[pre_gld_mask, "GLD"] = closes["GC=F"].loc[pre_gld_mask] *
gld_ratio * expense_factors_gld
    opens.loc[pre_gld_mask, "GLD"] = opens["GC=F"].loc[pre_gld_mask] *
gld_ratio * expense_factors_gld
```

[new](#)



```
highs.loc[pre_gld_mask, "GLD"] = highs["GC=F"].loc[pre_gld_mask] *
gld_ratio * expense_factors_gld
lows.loc[pre_gld_mask, "GLD"] = lows["GC=F"].loc[pre_gld_mask] *
gld_ratio * expense_factors_gld
volumes.loc[pre_gld_mask, "GLD"] = 1_000_000.0

# 2. Backcast TLT with VUSTX
tlt_series = closes["TLT"].dropna()
tlt_start = tlt_series.index[0]
tlt_anchor = tlt_series.loc[tlt_start]
vustx_anchor = closes["VUSTX"].asof(tlt_start)
if pd.isna(vustx_anchor) or vustx_anchor <= 0:
    vustx_anchor = closes["VUSTX"].dropna().iloc[0]
tlt_ratio = tlt_anchor / vustx_anchor
pre_tlt_mask = closes.index < tlt_start

closes.loc[pre_tlt_mask, "TLT"] = closes["VUSTX"].loc[pre_tlt_mask] *
tlt_ratio
opens.loc[pre_tlt_mask, "TLT"] = opens["VUSTX"].loc[pre_tlt_mask] *
tlt_ratio
highs.loc[pre_tlt_mask, "TLT"] = highs["VUSTX"].loc[pre_tlt_mask] *
tlt_ratio
lows.loc[pre_tlt_mask, "TLT"] = lows["VUSTX"].loc[pre_tlt_mask] *
tlt_ratio
volumes.loc[pre_tlt_mask, "TLT"] = 1_000_000.0

records = []
for tic in tickers:
    df_t = pd.DataFrame({
        "date": closes.index, "tic": tic, "open": opens[tic].values,
        "high": highs[tic].values, "low": lows[tic].values,
        "close": closes[tic].values, "volume": volumes[tic].values,
    })
    records.append(df_t)

panel_df = pd.concat(records, ignore_index=True)
panel_df["date"] = _ensure_naive_datetime(panel_df["date"])

macro_df = pd.DataFrame(index=closes.index)
macro_df["date"] = _ensure_naive_datetime(closes.index)
```

[new](#)



```
macro_df["vix_log"] =
np.log(closes["^VIX"].clip(lower=1.0)).ffill().bfill().values
macro_df["yield_curve_slope"] = (closes["^TNX"] -
closes["^IRX"]).ffill().bfill().values
macro_df["gold_logret"] =
np.log(closes["GC=F"].clip(lower=1e-3)).diff().clip(-0.2,
0.2).fillna(0.0).values
macro_df["wti_logret"] =
np.log(closes["CL=F"].clip(lower=1e-3)).diff().clip(-0.2,
0.2).fillna(0.0).values

try:
    fed = web.DataReader("FEDFUNDS", "fred", start_date,
end_date).reset_index()
    fed.columns = ["date", "fed_raw"]
    fed["date"] = _ensure_naive_datetime(fed["date"]) +
pd.Timedelta(days=CFG.fed_funds_lag_days)
    macro_df = pd.merge_asof(macro_df.sort_values("date"),
fed.sort_values("date"), on="date", direction="backward")
    macro_df["fed_funds_rate"] = macro_df["fed_raw"].fillna(1.0) /
100.0
except Exception:
    macro_df["fed_funds_rate"] = 0.02

full_df = pd.merge(panel_df, macro_df, on="date", how="inner")
return full_df.sort_values(["date", "tic"]).reset_index(drop=True)

#
=====
=====
# SECTION 2: INSTITUTIONAL FEATURE ENGINEERING (CROSS-SECTIONAL + MOM
RANK)
#
=====
=====

def compute_institutional_features(df: pd.DataFrame, tickers: Tuple[str,
...]) -> pd.DataFrame:
    df = df.copy().sort_values(["tic", "date"]).reset_index(drop=True)
    processed_tics = []
```

[new](#)



```
for tic, group in df.groupby("tic", sort=False):
    g = group.copy()
    close = g["close"].astype(float)

    # Technicals
    rsi = ta.momentum.RSIIndicator(close, window=14).rsi()
    g["RSI_14"] = (rsi / 100.0) - 0.5

    macd = ta.trend.MACD(close, window_slow=26, window_fast=12,
window_sign=9)
    ema26 = ta.trend.EMAIndicator(close,
window=26).ema_indicator().replace(0, np.nan)
    g["MACD_hist_pct"] = macd.macd_diff() / ema26

    ema12 = ta.trend.EMAIndicator(close, window=12).ema_indicator()
    ema50 = ta.trend.EMAIndicator(close, window=50).ema_indicator()
    ema200 = ta.trend.EMAIndicator(close,
window=200).ema_indicator().replace(0, np.nan)
    sma200 = close.rolling(200).mean().replace(0, np.nan)

    g["EMA_12_26_ratio"] = (ema12 - ema26) / ema26
    g["EMA_50_200_ratio"] = (ema50 - ema200) / ema200
    g["price_to_EMA200"] = (close - ema200) / ema200

    stoch = ta.momentum.StochRSIIndicator(close, window=14, smooth1=3,
smooth2=3)
    g["StochRSI_K"] = stoch.stochrsi_k() - 0.5

    # Multi-Horizon Momentum
    g["mom_20d"] = close.pct_change(20).clip(-0.5, 0.5)
    g["mom_63d"] = close.pct_change(63).clip(-0.5, 0.5)

    # Trend Regime Gate: Price < SMA200 and Momentum < 0
    g["downtrend_flag"] = ((close < sma200) & (g["mom_20d"] <
0)).astype(np.float32)
    processed_tics.append(g)

df = pd.concat(processed_tics, ignore_index=True)
```

[new](#)



```
# Relative RSI
pivoted_rsi = df.pivot(index="date", columns="tic", values="RSI_14")
mean_rsi = pivoted_rsi.mean(axis=1)
df["RSI_rel"] = df["RSI_14"] - df["date"].map(mean_rsi)

# Cross-Sectional Ratios
pivoted_close = df.pivot(index="date", columns="tic", values="close")
ratio_qqq_spy = (pivoted_close["QQQ"] /
pivoted_close["SPY"]).pct_change(20)
ratio_spy_tlt = (pivoted_close["SPY"] /
pivoted_close["TLT"]).pct_change(20)
df["ratio_QQQ_SPY"] = df["date"].map(ratio_qqq_spy).fillna(0.0)
df["ratio_SPY_TLT"] = df["date"].map(ratio_spy_tlt).fillna(0.0)

# Cross-Sectional 3-Month Momentum Percentile Rank
pivoted_mom63 = df.pivot(index="date", columns="tic",
values="mom_63d")
rank_df = (
    pivoted_mom63.rank(axis=1, pct=True)
    .reset_index()
    .melt(id_vars="date", value_name="mom_rank_63d")
)
df = pd.merge(df, rank_df, on=["date", "tic"], how="left")

return df.sort_values(["date", "tic"]).reset_index(drop=True)

#
=====
====
# SECTION 3: PURE VOLATILITY TARGETING OVERLAY (TIER 2 - CLEAN & ROBUST)
#
=====
=====

class VolatilityTargetingEngine:
    """
    Tier 2 Pure Deterministic Risk Overlay:
    - Dynamic Exposure Scalar via EWMA Volatility (lambda=0.94)
    - Continuous Sizing without Discrete Whipsaw Traps
```

new



```
- Soft Drawdown Dampener (Gradual deleveraging only at severe DD > 15%)
"""
def __init__(self, target_annual_vol: float = 0.085, ewma_lambda: float = 0.94):
    # ปรับ Target Vol เป็น 8.5% ต่อปี เพื่อคุม MaxDD ปี 2022 ให้กระชับขึ้น
    self.target_daily_vol = target_annual_vol / np.sqrt(252.0)
    self.decay = ewma_lambda
    self.variance = (self.target_daily_vol) ** 2
    self.last_scalar = 1.0

def reset(self):
    self.variance = (self.target_daily_vol) ** 2
    self.last_scalar = 1.0

def update_and_scale(self, portfolio_step_return: float, current_drawdown: float) -> Tuple[float, float]:
    # Recursive EWMA Realized Volatility
    self.variance = self.decay * self.variance + (1.0 - self.decay) * (portfolio_step_return ** 2)
    realized_vol = np.sqrt(max(self.variance, 1e-8))

    # Continuous Volatility Targeting
    raw_scalar = float(self.target_daily_vol / realized_vol)

    # Soft Drawdown Dampener: เมื่อเกิด Drawdown ลึก (>15%) ให้ชะลอ Exposure ลงแบบนุ่มนวล
    if current_drawdown > 0.15:
        dd_penalty = 1.0 - min(0.50, (current_drawdown - 0.15) * 2.0)
        raw_scalar *= dd_penalty

    applied_scalar = float(np.clip(raw_scalar, CFG.min_exposure, CFG.max_exposure))

    # Hysteresis Filter (5% buffer ป้องกัน Churning)
    if abs(applied_scalar - self.last_scalar) < CFG.hysteresis_band:
        applied_scalar = self.last_scalar
    else:
        self.last_scalar = applied_scalar
```

[new](#)



```
        return applied_scalar, realized_vol

#
=====
====
# SECTION 4: ENVIRONMENT (TIER 1 RELATIVE ALPHA + TREND REGIME GATE)
#
=====
====

class InstitutionalPortfolioEnv(gym.Env):
    metadata = {"render_modes": []}

    def __init__(self, df: pd.DataFrame, tickers: Tuple[str, ...],
is_training: bool = False):
        super().__init__()
        self.df = df.copy()
        self.tickers = tuple(tickers)
        self.k = len(self.tickers)
        self.is_training = is_training

        self.dates = sorted(self.df["date"].unique())
        self.total_days = len(self.dates)
        if self.total_days < 10:
            raise ValueError("Environment requires at least 10 trading
days.")

        self.risk_engine =
VolatilityTargetingEngine(CFG.target_annual_vol, CFG.vol_ewma_lambda)

        obs_dim = (self.k * len(ALL_FEATURES)) + (self.k + 1) + 2
        self.observation_space = spaces.Box(low=-np.inf, high=np.inf,
shape=(obs_dim,), dtype=np.float32)
        self.action_space = spaces.Box(low=-3.0, high=3.0,
shape=(self.k,), dtype=np.float32)

        self._build_matrices()

    def _build_matrices(self):
        self.open_matrix = (
```

[new](#)



```
        self.df.pivot(index="date", columns="tic", values="open")
        .reindex(columns=self.tickers).to_numpy(dtype=np.float64)
    )
    self.close_matrix = (
        self.df.pivot(index="date", columns="tic", values="close")
        .reindex(columns=self.tickers).to_numpy(dtype=np.float64)
    )

    ff = self.df.pivot(index="date", columns="tic",
values="fed_funds_rate").iloc[:, 0].to_numpy(dtype=np.float64)
    self.daily_cash_yield = (1.0 + ff) ** (1.0 / 252.0) - 1.0

    if "downtrend_flag" in self.df.columns:
        self.downtrend_matrix = (
            self.df.pivot(index="date", columns="tic",
values="downtrend_flag")
.reindex(columns=self.tickers).fillna(0.0).to_numpy(dtype=bool)
        )
    else:
        self.downtrend_matrix = np.zeros((self.total_days, self.k),
dtype=bool)

    feat_list = []
    for tic in self.tickers:
        t_df = self.df[self.df["tic"] == tic].sort_values("date")

feat_list.append(t_df[ALL_FEATURES].to_numpy(dtype=np.float32))
        self.feat_matrix = np.stack(feat_list, axis=1)

    def reset(self, seed=None, options=None):
        super().reset(seed=seed)
        self.risk_engine.reset()

    available_days = self.total_days - 2
    if self.is_training:
        min_len = min(252, max(30, available_days // 3))
        max_len = min(756, max(min_len, available_days))
        self.episode_length = random.randint(min_len, max_len)
        max_start = max(0, available_days - self.episode_length)
```

[new](#)



```
        self.start_day = random.randint(0, max_start)
        self.max_day = min(available_days, self.start_day +
self.episode_length)
    else:
        self.start_day = 0
        self.max_day = available_days

    self.current_day = self.start_day
    self.portfolio_value = float(CFG.initial_amount)
    self.peak_value = float(CFG.initial_amount)

    self.weights = np.zeros(self.k + 1, dtype=np.float64)
    self.weights[0] = 1.0
    self.obs_weights = self.weights.copy()

    self.asset_memory = [self.portfolio_value]
    self.date_memory = [self.dates[self.current_day + 1]]

    obs = self._get_obs(self.current_day, scalar=1.0,
realized_vol=self.risk_engine.target_daily_vol)
    info = {"date": self.dates[self.current_day], "portfolio_value":
self.portfolio_value}
    return obs, info

    def _get_obs(self, day_idx: int, scalar: float, realized_vol: float)
-> np.ndarray:
        feats = self.feats_matrix[day_idx].reshape(-1)
        risk_feats = np.array([scalar, realized_vol /
self.risk_engine.target_daily_vol], dtype=np.float32)
        return np.concatenate([feats, self.obs_weights,
risk_feats]).astype(np.float32)

    @staticmethod
    def _softmax(actions: np.ndarray) -> np.ndarray:
        # ใช้ Standard Softmax (tau=1.0) เพื่อคุม Turnover ไม่ให้เกิน 1.5% ต่อวัน
        shifted = actions - np.max(actions)
        exp_a = np.exp(shifted)
        return exp_a / np.sum(exp_a)

    def step(self, actions: np.ndarray):
```

new



```
raw_risky_weights = self._softmax(actions)

current_dd = max(0.0, 1.0 - (self.portfolio_value /
self.peak_value))

# Tier 2: Dynamic Volatility Targeting
prev_step_ret = 0.0 if len(self.asset_memory) < 2 else
(self.asset_memory[-1] / self.asset_memory[-2] - 1.0)
scalar, realized_vol =
self.risk_engine.update_and_scale(prev_step_ret, current_dd)

final_risky_weights = raw_risky_weights * scalar
cash_weight = max(0.0, 1.0 - float(np.sum(final_risky_weights)))
target_weights = np.concatenate([[cash_weight],
final_risky_weights])
target_weights /= np.sum(target_weights)

# No-Trade Band Filter
turnover_candidate = float(np.sum(np.abs(target_weights[1:] -
self.weights[1:])))
if turnover_candidate < CFG.min_turnover_threshold:
    target_weights = self.weights.copy()
    turnover = 0.0
    rebalance_cost = 0.0
else:
    turnover = turnover_candidate
    rebalance_cost = float(self.portfolio_value * turnover *
CFG.transaction_cost_pct)

exec_open = self.open_matrix[self.current_day + 1]
next_open = self.open_matrix[self.current_day + 2]

net_capital = self.portfolio_value - rebalance_cost
cash_allocated = net_capital * target_weights[0]
risky_allocated = net_capital * target_weights[1:]

cash_earned = cash_allocated * (1.0 +
self.daily_cash_yield[self.current_day + 1])
risky_earned = risky_allocated * (next_open / exec_open)
```

[new](#)



```
new_asset_values = np.concatenate([[cash_earned], risky_earned])
new_portfolio_value = float(np.sum(new_asset_values))

step_return = (new_portfolio_value - self.portfolio_value) /
self.portfolio_value
valuation_weights = new_asset_values / new_portfolio_value

# Benchmark Return: เทียบกับ EqW ดิบใน Reward เพื่อผลักดันให้ AI แข่งกับตลาด
เต็มตัว
eqw_return = float(np.mean(next_open / exec_open) - 1.0)
excess_return = step_return - eqw_return

# Clean Objective: Excess Return ลบ Turnover Cost (ไม่มีค่าคงที่ติดลบมา
บกวณ)
reward = float((excess_return * 100.0) - (turnover *
CFG.transaction_cost_pct * 100.0))

self.portfolio_value = new_portfolio_value
self.weights = valuation_weights
self.peak_value = max(self.peak_value, self.portfolio_value)

close_today = self.close_matrix[self.current_day + 1]
drifted_risky = risky_allocated * (close_today / exec_open)
drifted_total = cash_allocated + np.sum(drifted_risky)
self.obs_weights = np.concatenate([[cash_allocated],
drifted_risky]) / drifted_total

self.current_day += 1
truncated = self.current_day >= self.max_day
terminated = False

val_date = self.dates[self.current_day + 1]
self.asset_memory.append(self.portfolio_value)
self.date_memory.append(val_date)

info = {
    "date": val_date,
    "portfolio_value": self.portfolio_value,
    "step_return": step_return,
    "turnover": turnover,
```

[new](#)



```
        "transaction_cost": rebalance_cost,
        "scalar": scalar,
        "realized_vol": realized_vol,
        "weights": valuation_weights,
        "excess_return": excess_return,
    }

    obs = self._get_obs(self.current_day, scalar=scalar,
realized_vol=realized_vol)
    return obs, reward, terminated, truncated, info
#
=====
=====
# SECTION 5: DATA PURGING & TRAIN-ONLY SCALING
#
=====
=====

def clean_and_align_dataset(df: pd.DataFrame, tickers: Tuple[str, ...]) ->
pd.DataFrame:
    df = df.copy().sort_values(["tic", "date"]).reset_index(drop=True)
    for col in ALL_FEATURES + ["open", "close"]:
        df[col] = df.groupby("tic")[col].ffill()

    df = df.dropna(subset=ALL_FEATURES + ["open", "close"]).copy()
    counts = df.groupby("date")["tic"].nunique()
    valid_dates = counts[counts == len(tickers)].index
    df = df[df["date"].isin(valid_dates)].copy()

    for col in ALL_FEATURES:
        df[col] = df[col].replace([np.inf, -np.inf], np.nan)
    df = df.dropna(subset=ALL_FEATURES).copy()

    print(f">> Dataset Cleaned: เหลือ {len(valid_dates)} วันทำการที่ข้อมูลสมบูรณ์
100% "
        f"({valid_dates[0].date()} ถึง {valid_dates[-1].date()})")
    return df.sort_values(["date", "tic"]).reset_index(drop=True)

def compute_scaling_stats(train_df: pd.DataFrame) -> Dict[str,
Tuple[float, float]]:
```

[new](#)



```
stats = {}
for col in ALL_FEATURES:
    series = train_df[col].dropna()
    mean = float(series.mean())
    std = float(series.std())
    stats[col] = (mean, std if std > 1e-6 else 1.0)
return stats

def apply_scaling(df: pd.DataFrame, stats: Dict[str, Tuple[float, float]],
clip_sigma: float = 5.0) -> pd.DataFrame:
    df = df.copy()
    for col, (mean, std) in stats.items():
        if col in df.columns:
            df[col] = ((df[col] - mean) / std).clip(-clip_sigma,
clip_sigma)
    return df

#
=====
====
# SECTION 6: STATISTICAL METRICS & DEFLATED SHARPE
#
=====
====

def compute_deflated_sharpe_ratio(estimated_sharpe: float, n_trials: int,
returns: np.ndarray) -> float:
    n = len(returns)
    if n < 30 or np.std(returns) == 0:
        return 0.0
    skew = float(stats.skew(returns))
    kurt = float(stats.kurtosis(returns, fisher=True)) + 3.0
    em_constant = 0.5772156649
    expected_max_sr = (1.0 - em_constant) * stats.norm.ppf(1.0 - 1.0 /
n_trials) + em_constant * stats.norm.ppf(1.0 - 1.0 / (n_trials * np.e))
    sr_std = np.sqrt((1.0 + 0.5 * (estimated_sharpe ** 2) - skew *
estimated_sharpe + ((kurt - 3.0) / 4.0) * (estimated_sharpe ** 2)) / (n -
1.0))
    z_stat = (estimated_sharpe - expected_max_sr) / max(sr_std, 1e-6)
    return float(stats.norm.cdf(z_stat))
```

[new](#)



```
def compute_sortino_ratio(returns: np.ndarray, target: float = 0.0,
periods_per_year: float = 252.0) -> float:
    downside = returns[returns < target] - target
    downside_dev = np.sqrt(np.mean(downside ** 2)) if len(downside) > 0
else 0.0
    if downside_dev == 0:
        return 0.0
    return float((np.mean(returns) - target) / downside_dev *
np.sqrt(periods_per_year))

def compute_extended_trade_stats(returns: np.ndarray) -> Dict[str, float]:
    """
    หมายเหตุ: ระบบนี้ rebalance แบบต่อเนื่องทุกวัน ไม่มี "trade" เข้า-ออกแบบดั้งเดิม
    Win Rate / Profit Factor / Risk-Reward / Expectancy จึงคำนวณจาก daily
step_return
    เป็น proxy เทียบเท่าสถิติการเทรด ไม่ใช่การนับจำนวน trade จริง
    """
    wins = returns[returns > 0]
    losses = returns[returns < 0]
    win_rate = (len(wins) / len(returns) * 100.0) if len(returns) > 0 else
0.0
    gross_profit = float(np.sum(wins))
    gross_loss = float(np.abs(np.sum(losses)))
    profit_factor = (gross_profit / gross_loss) if gross_loss > 0 else
float("inf")
    avg_win = float(np.mean(wins)) if len(wins) > 0 else 0.0
    avg_loss = float(np.mean(np.abs(losses))) if len(losses) > 0 else 0.0
    risk_reward = (avg_win / avg_loss) if avg_loss > 0 else float("inf")
    expectancy = (win_rate / 100.0 * avg_win) - ((1.0 - win_rate / 100.0)
* avg_loss)
    return {
        "win_rate": win_rate, "profit_factor": profit_factor,
        "avg_win": avg_win, "avg_loss": avg_loss,
        "risk_reward": risk_reward, "expectancy": expectancy,
    }

def run_ablation_baselines(test_df: pd.DataFrame, tickers: Tuple[str,
...]) -> Dict[str, pd.Series]:
```

[new](#)



```
open_piv = test_df.pivot(index="date", columns="tic",
values="open").reindex(columns=tickers).dropna()
dates = open_piv.index
n_days = len(dates)

eqw_returns = []
eqw_vt_returns = []
vt_engine = VolatilityTargetingEngine(target_annual_vol=0.085)

for i in range(1, n_days - 1):
    rel = open_piv.iloc[i + 1].values / open_piv.iloc[i].values
    raw_ret = float(np.mean(rel) - 1.0)
    eqw_returns.append(raw_ret)

    # Volatility Targeting อีกระหว่าง EqW
    scalar, _ =
vt_engine.update_and_scale(portfolio_step_return=raw_ret,
current_drawdown=0.0)
    eqw_vt_returns.append(scalar * raw_ret)

return {
    "EqW": pd.Series(eqw_returns, index=dates[2:]),
    "EqW-VT": pd.Series(eqw_vt_returns, index=dates[2:])
}

#
=====
====
# SECTION 7: AUDIT SUITE
#
=====
====

def run_institutional_audit_suite():
    print("\n" + "=" * 78)
    print(" PPO_V3.1 INSTITUTIONAL FINANCIAL & MATHEMATICAL AUDIT SUITE")
    print("=" * 78)

    dates = pd.bdate_range("2025-01-01", periods=15)
    rows = []
```

[new](#)



```
for day, date in enumerate(dates):
    for j, tic in enumerate(CFG.core_tickers):
        row = {
            "date": date, "tic": tic, "open": 100.0 + day * 1.5,
            "high": 102.0 + day * 1.5, "low": 99.0 + day * 1.5,
            "close": 101.0 + day * 1.5, "volume": 100_000.0,
"fed_funds_rate": 0.05,
            "downtrend_flag": 0.0
        }
        for feat in ALL_FEATURES:
            row[feat] = 0.01 * (j + 1)
        rows.append(row)
synth_df = pd.DataFrame(rows)

env = InstitutionalPortfolioEnv(synth_df, CFG.core_tickers,
is_training=False)
obs, info = env.reset()
assert np.isclose(env.portfolio_value, CFG.initial_amount)
assert np.isclose(env.weights[0], 1.0)
print(" PASS: 1. Environment Accounting & Cash Yield Initialization")

action = np.zeros(len(CFG.core_tickers), dtype=np.float32)
obs, reward, term, trunc, info = env.step(action)
assert np.isclose(np.sum(info["weights"]), 1.0, atol=1e-6)
print(" PASS: 2. Two-Tier Deterministic Volatility Targeting Math")

assert np.isclose(np.sum(env.obs_weights), 1.0, atol=1e-5)
print(" PASS: 3. Zero-Leakage Drift Parity at Close(t+1)")

# Test Multi-Stage Circuit Breaker
# Test Pure Volatility Targeting Math & Soft Dampener
engine = VolatilityTargetingEngine(target_annual_vol=0.085)
s1, _ = engine.update_and_scale(0.0, current_drawdown=0.05)
assert s1 <= 1.0, "Normal scalar must respect leverage cap"
s2, _ = engine.update_and_scale(0.0, current_drawdown=0.25)
assert s2 < s1, f"Severe drawdown must damp exposure smoothly (Got
s1={s1:.3f}, s2={s2:.3f})"
print(" PASS: 4. Pure Volatility Targeting Math & Continuous Sizing")
print("=" * 78)
print(" ALL INSTITUTIONAL AUDITS PASSED SUCCESSFULLY.")
```

[new](#)



```
print("=" * 78 + "\n")

#
=====

====

# SECTION 8: WALK-FORWARD PIPELINE & REPORTERS
#
=====

====

class EnvFactory:
    def __init__(self, df: pd.DataFrame, tickers: Tuple[str, ...], rank:
int, base_seed: int, is_training: bool):
        self.df = df
        self.tickers = tickers
        self.rank = rank
        self.base_seed = base_seed
        self.is_training = is_training

    def __call__(self):
        env = InstitutionalPortfolioEnv(self.df, self.tickers,
is_training=self.is_training)
        env.reset(seed=self.base_seed + self.rank)
        return env

def run_production_walk_forward(raw_df: pd.DataFrame, out_dir: Path):
    out_dir.mkdir(parents=True, exist_ok=True)
    all_oos_accounts = []
    fold_summaries = []

    for fold_id, (tr_s, tr_e, v_s, v_e, te_s, te_e) in
enumerate(WALK_FORWARD_FOLDS, start=1):
        print("\n" + "=" * 78)
        print(f"SINGLE HOLDOUT SPLIT | Train [{tr_s}:{tr_e}] | Val
[{v_s}:{v_e}] | Test [{te_s}:{te_e}]")
        print("=" * 78)

        train_slice = raw_df[(raw_df["date"] >= tr_s) & (raw_df["date"] <=
tr_e)].copy()
```

[new](#)



```
val_slice = raw_df[(raw_df["date"] >= v_s) & (raw_df["date"] <=
v_e)].copy()
test_slice = raw_df[(raw_df["date"] >= te_s) & (raw_df["date"] <=
te_e)].copy()

if train_slice.empty or val_slice.empty or test_slice.empty:
    continue

tr_dates = sorted(train_slice["date"].unique())
v_dates = sorted(val_slice["date"].unique())
te_dates = sorted(test_slice["date"].unique())
print(f"TRAIN: {tr_dates[0].strftime('%Y-%m-%d')} ->
{tr_dates[-1].strftime('%Y-%m-%d')} ({len(tr_dates)} วันทำการ)")
print(f"VAL : {v_dates[0].strftime('%Y-%m-%d')} ->
{v_dates[-1].strftime('%Y-%m-%d')} ({len(v_dates)} วันทำการ)")
print(f"TEST : {te_dates[0].strftime('%Y-%m-%d')} ->
{te_dates[-1].strftime('%Y-%m-%d')} ({len(te_dates)} วันทำการ)")

scaling_stats = compute_scaling_stats(train_slice)
train_df = apply_scaling(train_slice, scaling_stats)
val_df = apply_scaling(val_slice, scaling_stats)
test_df = apply_scaling(test_slice, scaling_stats)

fold_path = out_dir / f"fold_{fold_id}"
fold_path.mkdir(exist_ok=True)

env_fns = [
    EnvFactory(train_df, CFG.core_tickers, i, CFG.seed + fold_id *
100, is_training=True)
    for i in range(CFG.max_cpu)
]
env_train = VecNormalize(
    SubprocVecEnv(env_fns, start_method="spawn"),
    norm_obs=False, norm_reward=True, gamma=CFG.gamma
)
env_val = VecNormalize(
    DummyVecEnv([lambda: InstitutionalPortfolioEnv(val_df,
CFG.core_tickers, is_training=False)]),
    norm_obs=False, norm_reward=False, training=False
)
```

[new](#)



```
policy_kwargs = dict(net_arch=dict(pi=[128, 128], vf=[128, 128]),
activation_fn=nn.Tanh, ortho_init=True)
model = PPO(
    "MlpPolicy", env_train, learning_rate=CFG.learning_rate,
    n_steps=CFG.target_rollout // CFG.max_cpu,
batch_size=CFG.batch_size,
    n_epochs=CFG.n_epochs, gamma=CFG.gamma,
gae_lambda=CFG.gae_lambda,
    ent_coef=CFG.ent_coef, vf_coef=CFG.vf_coef,
clip_range=CFG.clip_range,
    max_grad_norm=CFG.max_grad_norm, target_kl=CFG.target_kl,
    policy_kwargs=policy_kwargs, seed=CFG.seed + fold_id,
device=CFG.device, verbose=0
)

eval_cb = EvalCallback(
    env_val, best_model_save_path=str(fold_path),
    eval_freq=max(500, 2000 // CFG.max_cpu), deterministic=True,
verbose=0
)

try:
    model.learn(total_timesteps=CFG.total_timesteps,
callback=eval_cb)
finally:
    model.save(str(fold_path / "last_model.zip"))
    env_train.close()
    env_val.close()

best_model_file = fold_path / "best_model.zip"
active_model = PPO.load(str(best_model_file if
best_model_file.exists() else (fold_path / "last_model.zip")),
device=CFG.device)

env_test = InstitutionalPortfolioEnv(test_df, CFG.core_tickers,
is_training=False)
obs, _ = env_test.reset()
records = []
```

[new](#)



```
while True:
    action, _ = active_model.predict(obs, deterministic=True)
    obs, reward, terminated, truncated, info =
env_test.step(action)
    records.append({
        "date": info["date"], "portfolio_value":
info["portfolio_value"],
        "step_return": info["step_return"], "scalar":
info["scalar"],
        "turnover": info["turnover"], "cash_weight":
float(info["weights"][0]),
        "eqw_matched_return": info.get("eqw_matched_return",
info["step_return"]),
    })
    if terminated or truncated:
        break

df_oos = pd.DataFrame(records)
df_oos.to_csv(fold_path / "oos_account.csv", index=False)
all_oos_accounts.append(df_oos)

ai_ret = df_oos["step_return"].values
ai_days = len(ai_ret)
ai_cagr = ((df_oos["portfolio_value"].iloc[-1] /
df_oos["portfolio_value"].iloc[0]) ** (252.0 / ai_days) - 1.0) * 100.0
ai_cum = np.cumprod(1.0 + ai_ret)
ai_peak = np.maximum.accumulate(ai_cum)
ai_max_dd = np.min((ai_cum - ai_peak) / ai_peak) * 100.0
ai_std = np.std(ai_ret, ddof=1)
ai_sharpe = (np.mean(ai_ret) / ai_std * np.sqrt(252.0)) if ai_std
> 0 else 0.0
ai_avg_cash = float(df_oos["cash_weight"].mean()) * 100.0
ai_sortino = compute_sortino_ratio(ai_ret)
ai_stats = compute_extended_trade_stats(ai_ret)

# 4. คำนวณ Performance ราย Fold ของ Baselines
baselines = run_ablation_baselines(test_df, CFG.core_tickers)

# 1. EqW-Raw
eqw_ret = baselines["EqW"].values
```

[new](#)



```
eqw_cagr = ((np.cumprod(1.0 + eqw_rets)[-1]) ** (252.0 /
len(eqw_rets)) - 1.0) * 100.0
eqw_max_dd = np.min((np.cumprod(1.0 + eqw_rets) -
np.maximum.accumulate(np.cumprod(1.0 + eqw_rets))) /
np.maximum.accumulate(np.cumprod(1.0 + eqw_rets))) * 100.0
eqw_std = np.std(eqw_rets, ddof=1)
eqw_sharpe = (np.mean(eqw_rets) / eqw_std * np.sqrt(252.0)) if
eqw_std > 0 else 0.0

# 2. EqW-VT (Matched Volatility อีสระ)
eqw_vt_rets = baselines["EqW-VT"].values
eqw_vt_cagr = ((np.cumprod(1.0 + eqw_vt_rets)[-1]) ** (252.0 /
len(eqw_vt_rets)) - 1.0) * 100.0
eqw_vt_max_dd = np.min((np.cumprod(1.0 + eqw_vt_rets) -
np.maximum.accumulate(np.cumprod(1.0 + eqw_vt_rets))) /
np.maximum.accumulate(np.cumprod(1.0 + eqw_vt_rets))) * 100.0
eqw_vt_std = np.std(eqw_vt_rets, ddof=1)
eqw_vt_sharpe = (np.mean(eqw_vt_rets) / eqw_vt_std *
np.sqrt(252.0)) if eqw_vt_std > 0 else 0.0

print(f"AI      : CAGR {ai_cagr:7.2f}% | MaxDD {ai_max_dd:7.2f}% |
Sharpe {ai_sharpe:6.2f} | AvgCash {ai_avg_cash:6.2f}%")
print(f"EqW-VT  : CAGR {eqw_vt_cagr:7.2f}% | MaxDD
{eqw_vt_max_dd:7.2f}% | Sharpe {eqw_vt_sharpe:6.2f} (Matched Risk)")
print(f"EqW-Raw : CAGR {eqw_cagr:7.2f}% | MaxDD {eqw_max_dd:7.2f}%
| Sharpe {eqw_sharpe:6.2f} (Unhedged 100%)")
print(f"ส่วนต่าง Pure Alpha (vs EqW-VT) : {ai_cagr -
eqw_vt_cagr:+.2f}% CAGR")
print(f"ส่วนต่าง Excess Beta (vs EqW-Raw): {ai_cagr -
eqw_cagr:+.2f}% CAGR")
print(f"AI Extra: Sortino {ai_sortino:6.2f} | WinRate
{ai_stats['win_rate']:6.2f}% | ProfitFactor
{ai_stats['profit_factor']:6.2f} | R:R {ai_stats['risk_reward']:6.2f} |
Expectancy {ai_stats['expectancy']:+.4f}%")

fold_summaries.append({
    "cagr": ai_cagr, "max_dd": ai_max_dd, "sharpe": ai_sharpe,
    "avg_cash": ai_avg_cash, "avg_turnover":
float(df_oos["turnover"].mean()),
    "sortino": ai_sortino, "win_rate": ai_stats["win_rate"],
```

[new](#)



```
        "profit_factor": ai_stats["profit_factor"], "risk_reward":
ai_stats["risk_reward"],
        "expectancy": ai_stats["expectancy"],
    })

summary_df = pd.DataFrame(fold_summaries)
print("\n" + "=" * 78)
print(" สรุปภาพรวม SINGLE HOLDOUT SPLIT VALIDATION")
print("=" * 78)
print(f"AI เฉ็ลย CAGR      : {summary_df['cagr'].mean():.2f}%")
print(f"AI เฉ็ลย MaxDD     : {summary_df['max_dd'].mean():.2f}%")
print(f"AI เฉ็ลย Sharpe      : {summary_df['sharpe'].mean():.2f}%")
print(f"AI Sharpe SD        : {summary_df['sharpe'].std(ddof=1):.2f}%")
print(f"AI เฉ็ลยถ็อเงินสด    : {summary_df['avg_cash'].mean():.2f}%")
print(f"AI เฉ็ลย Turnover    : {summary_df['avg_turnover'].mean():.4f}%")
print(f"AI เฉ็ลย Sortino     : {summary_df['sortino'].mean():.2f}%")
print(f"AI เฉ็ลย WinRate      : {summary_df['win_rate'].mean():.2f}%")
print(f"AI เฉ็ลย ProfitFactor:
{summary_df['profit_factor'].replace([np.inf, -np.inf],
np.nan).mean():.2f}")
    print(f"AI เฉ็ลย Risk:Reward :
{summary_df['risk_reward'].replace([np.inf, -np.inf],
np.nan).mean():.2f}")
    print(f"AI เฉ็ลย Expectancy :
{summary_df['expectancy'].mean():+.4f}%")

    stitched_oos = (
        pd.concat(all_oos_accounts, ignore_index=True)

.drop_duplicates("date").sort_values("date").reset_index(drop=True)
    )
    stitched_oos["daily_return"] = stitched_oos["step_return"]
    rl_returns = stitched_oos["daily_return"].values
    total_days = len(stitched_oos)

    cagr = ((stitched_oos["portfolio_value"].iloc[-1] /
stitched_oos["portfolio_value"].iloc[0]) ** (252.0 / total_days) - 1.0) *
100.0
    vol = np.std(rl_returns, ddof=1) * np.sqrt(252.0) * 100.0
```

[new](#)



```
    sharpe = (np.mean(rl_returns) / np.std(rl_returns, ddof=1)) *
np.sqrt(252.0) if np.std(rl_returns) > 0 else 0.0
    cum = np.cumprod(1.0 + rl_returns)
    peak = np.maximum.accumulate(cum)
    max_dd = np.min((cum - peak) / peak) * 100.0
    dsr_val = compute_deflated_sharpe_ratio(sharpe,
CFG.pre_registered_trials, rl_returns)
    stitched_sortino = compute_sortino_ratio(rl_returns)
    stitched_stats = compute_extended_trade_stats(rl_returns)

    print("\n" + "=" * 78)
    print(" MASTER STITCHED OUT-OF-SAMPLE PERFORMANCE (2016 - 2026)")
    print("=" * 78)
    print(f" Annualized Return (CAGR) : {cagr:7.2f}%")
    print(f" Annualized Volatility : {vol:7.2f}%")
    print(f" Maximum Drawdown (MaxDD) : {max_dd:7.2f}%")
    print(f" Stitched Sharpe Ratio : {sharpe:7.2f}")
    print(f" Stitched Sortino Ratio : {stitched_sortino:7.2f}")
    print(f" Win Rate :
{stitched_stats['win_rate']:6.2f}%")
    print(f" Profit Factor :
{stitched_stats['profit_factor']:7.2f}")
    print(f" Risk-Reward Ratio :
{stitched_stats['risk_reward']:7.2f}")
    print(f" Expectancy (per day) :
{stitched_stats['expectancy']:+.4f}%")
    print(f" Deflated Sharpe (DSR) : {dsr_val:7.2f} (Trials =
{CFG.pre_registered_trials})")
    print("=" * 78)

if __name__ == "__main__":
    set_global_seed(CFG.seed)
    run_institutional_audit_suite()
    raw_market_data = fetch_and_stitch_universe(CFG.core_tickers,
CFG.global_fetch_start, CFG.global_end)
    raw_features_df = compute_institutional_features(raw_market_data,
CFG.core_tickers)
    clean_features_df = clean_and_align_dataset(raw_features_df,
CFG.core_tickers)
    output_directory = Path("./production_v3_1_results")
```

[new](#)



```
run_production_walk_forward(clean_features_df, output_directory)
```

new



new